

浙江大学实验报告

专业：微电子科学与工程
姓名：赵楼晗
学号：3240104430
日期：2026 年 5 月 11 日
地点：紫金港东 4-216

课程名称：电子电路设计实验 I 指导老师：施红军、叶险峰、邓靖靖 成绩：
实验名称：Arduino 实时音频处理实验 实验类型：设计型实验 同组学生姓名：徐莘宁

一、实验目的

- 1、学习掌握用 Arduino UNO 进行实时音频信号处理；
- 2、学习掌握 PCB 电路板的设计和制作；
- 3、熟悉电流信号转换成电压信号的原理、设计方法；
- 4、巩固电路原理图的绘制方法与规范，提高电路分析与设计能力；
- 5、巩固 PCB 布局布线的基本原则与设计流程，掌握元器件封装选择、设计规则检查等关键环节；
- 6、通过完整的原理图与 PCB 设计实践，培养工程化设计思维与硬件调试能力。

二、实验任务

2.1 电流电压转换实验

1. 熟悉电流-电压转换电路的工作原理，并设计出一个电路，将标准电流信号 $4\text{mA} \sim 20\text{mA}$ 转换为标准电压信号 $0\text{V} \sim 10\text{V}$ ，误差控制在 5% 以内。
2. 掌握原理图和 PCB 设计的基本流程，巩固焊接与电路调试。

2.2 Arduino 实时音频处理实验

1. 用 Arduino UNO 实现音频信号处理设计，在（1）音频正弦波信号源（2）混响发生器中选择一个电路，并分别加上（3）音频输出放大器，形成一个完整的电路，完成相应器件的选择，制作 Arduino UNO 扩展板。
2. 编制与调试相关程序。
3. 将制作的扩展板与 Arduino UNO 板组装后，进行系统联调。

三、实验原理

3.1 电流电压转换实验

在自动控制技术中，传感器输出的标准电流信号通常为 $4\text{mA} \sim 20\text{mA}$ ，为了便于进一步处理，需要将其转换为 $0\text{V} \sim 10\text{V}$ 的电压信号。这可以通过工作在线性区的运算放大器来实现。

如图1是一种常见的电流电压转换电路设计思路，其主要由两个运放组成的两级放大电路实现：

第一级放大电路：电阻 R_1 跨接在电流源两端，运放 A_1 采用差分输入，将 $4\text{mA} \sim 20\text{mA}$ 的电流转换为电压。此级放大电路的放大倍数为 1，输出电压 $V_{o1} = -I_i \times R_1$ ，实现了电流到电压的初步转换。

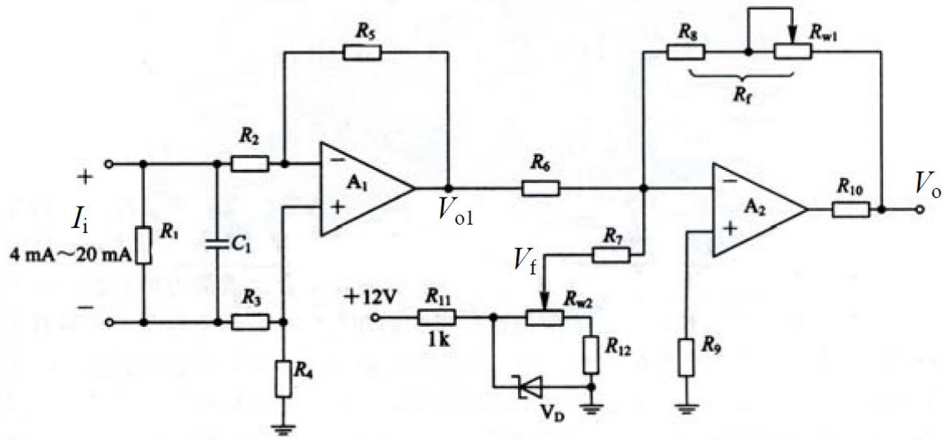


图 1: 电流电压转换电路样例

第二级放大电路：实现从 V_{o1} 到 $0\text{V} \sim 10\text{V}$ 的电平变换。根据对第二级电路的分析有：

$$\frac{V_{o1}}{R_6} + \frac{V_f}{R_7} = -\frac{V_o}{R_f}$$

结合两级运放的放大关系，可以推出输出电压 V_o 与输入电流 I_i 成线性关系：

$$V_o = \frac{R_f R_1 I_i}{R_6} - \frac{R_f V_f}{R_7}$$

因此，通过合理选取 R_6 、 R_7 的阻值，并调整 V_f 和 R_f 的值，可以确保当输入电流 I_i 从 4mA 变化到 20mA 时，输出电压 V_o 从 0V 变化到 10V 。

3.2 Arduino 实时音频处理实验

我和同组同学选择混响发生器与音频放大器两个电路进行探究。主要涉及以下原理：

3.2.1 混响发生器输入级

单片机中通过 ADC 采样将模拟信号数字化。Arduino 的 ADC 工作在输入电压 0 和 5V 之间，所以我们需要加入一个 2.5V 的偏置电压到输入信号上。通过耦合电容，将调整后在 0 至 5V 的范围内的信号发送到 ADC 输入端。图2为用于此目的的简单电路。

3.2.2 混响发生器中单片机的信号处理

在微控制器中，混响效果可以使用循环缓冲存储器实现，这个数字化的值存储在数组中。为了产生混响效果所需的延迟，单片机会反复读取循环缓冲区；延时用一单独的电位器调节。

如图3，电位器值和模拟输入信号都由中断程序采样。为此，交替地读出 A1（模拟输入）和 A0（电位器）。在主程序中，循环缓冲器被写入并在延迟后读取，时间延迟与 A0 处的电位器电压成比例。

3.2.3 音频放大电路

采用 LM386 运放实现有源音频放大电路，如图4；除了放大器，电路还具有音频滤波器的功能。 $2.2\mu\text{F}$ 电容器负责放大器的直流去耦，并且具有一定的高通特性，而 $1\mu\text{F}$ 电容器提供低通滤波。当使用简单的脉宽调制产生音

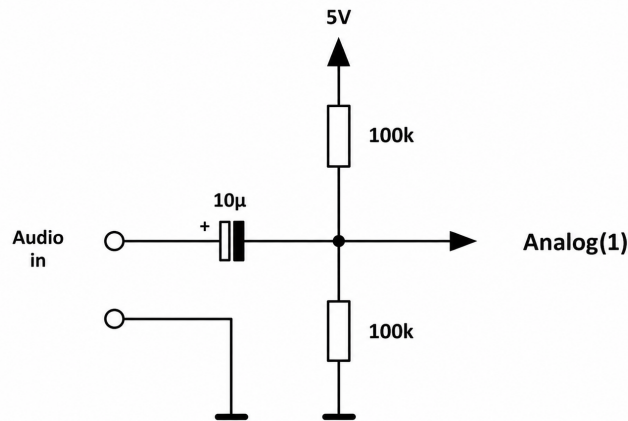


图 2: 输入级电路

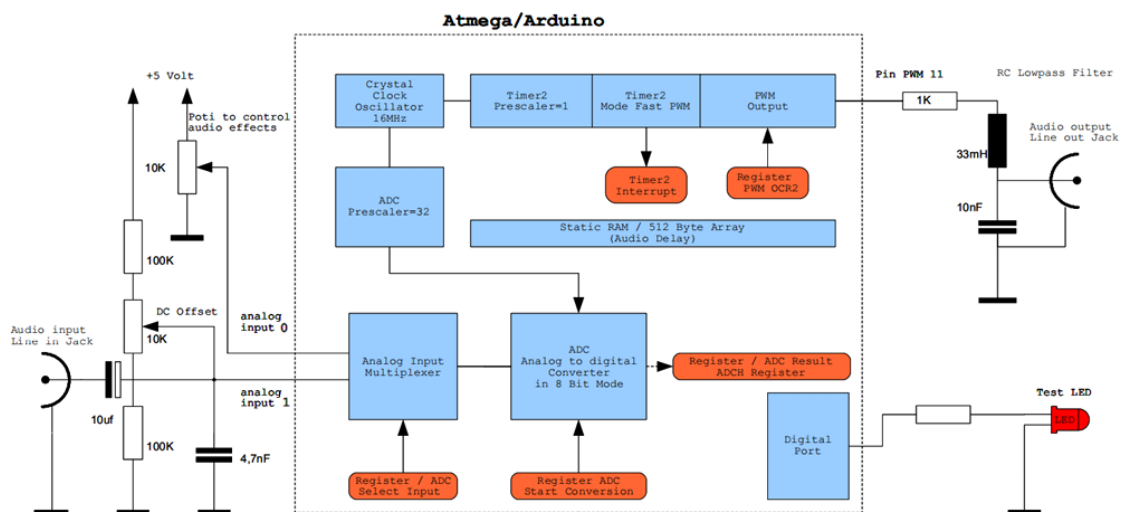


图 3: 基于中断的采样、信号处理

频信号时，高频 PWM 谐波将被很好地抑制，而在较低的千赫范围内所需的音频频率传递到放大器。

四、 主要仪器设备

主要使用的仪器有：Arduino Uno 开发板、加工所得的 PCB 印刷电路板、麦克风与扬声器元器件、示波器、信号发生器、若干电阻电容等电路元器件。

主要使用的软件有：Altium Designer 9、Arduino IDE、嘉立创 EDA 等。

五、 设计思路分析

在设计报告中已有详细的分析，此处仅作简短阐述。

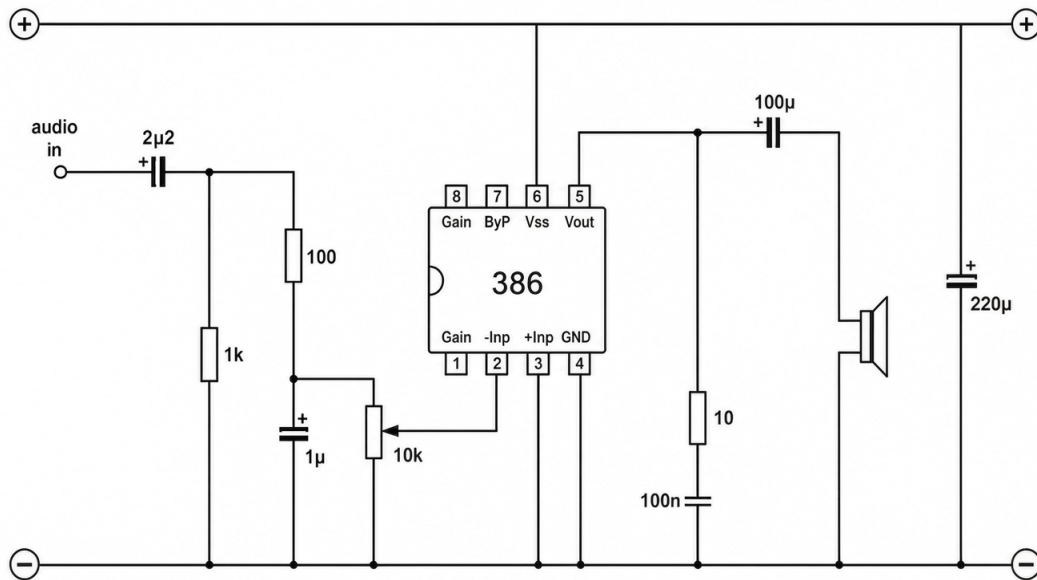


图 4: 音频放大电路

5.1 电流电压转换实验

原理图设计上，主要参考图1电路，按照两级电路进行设计：输入端 $4\text{mA} \sim 20\text{mA}$ 的电流通过第一级运放与电阻 $R_1 = 250\Omega$ 实现从电流信号到 $-1\text{V} \sim -5\text{V}$ 的电压信号的转换；输出的电压信号经过第二级运放再次进行反相放大，并通过电压耦合，最终输出要求的 $0\text{V} \sim 10\text{V}$ 的电压信号。最终设计的原理图参考图5。

PCB 设计上，主要根据老师给出的样例模仿学习。整体 PCB 设计的流程为：先在禁止布线层画出 PCB 的外轮廓线；再在原理图中导入元器件的封装及网表，对应到 PCB 中的飞线；接着合理安排元器件的位置使飞线的重叠与交叉尽可能少；最后选择层通过交互式布线连接所有器件，并通过 DRC 等后续检查验证设计。

5.2 Arduino 实时音频处理实验

原理图设计上，主要分成三级进行设计：输入级由麦克风采集音频信号并转化为电流，同时对于输入的音频信号先做一部分预处理，通过耦合电容过滤掉一部分低频噪声等；单片机级通过模拟端口读取音频信号（也就是采样），在内部进行数字信号处理（我们组选择的是混叠，具体来说就是将一部分信号经延迟后与原信号叠加，实现类似回声的效果），最终通过数模转换后输出；最终输出级为一个 LM386 运放构成的有源放大电路，可将音频信号放大作为最终的输出。最终设计的电路如图6。

程序设计部分，主要按照 ADC 采样 $\Rightarrow$ 环形缓冲区 $\Rightarrow$ PWM 输出三级进行设计：ADC 采样部分通过定义 A0 引脚读取音频的模拟信号，通过 A1 引脚读取电位器的电平决定混响的延迟时间；环形缓冲区部分用 512 字节缓冲区存储延迟采样，实现混响效果；输出部分通过定义 11 脚作为 PWM 输出口，通过可调占空比实现模拟信号的效果。

PCB 设计部分，整体按照飞线交叉最少的原则进行，基本按照原理图中的三级电路进行元器件布局并连线；布线时考虑到信号的传输问题与加工工艺，采用电源线宽 50mil 、信号线宽 20mil ，同时在走线时尽可能短，以及避免直角转弯，以保证信号之间基本不会干扰、保证了传输完整性。

六、 原理图设计

电流电压转换实验原理图设计如图5；Arduino 音频处理实验原理图如图6。

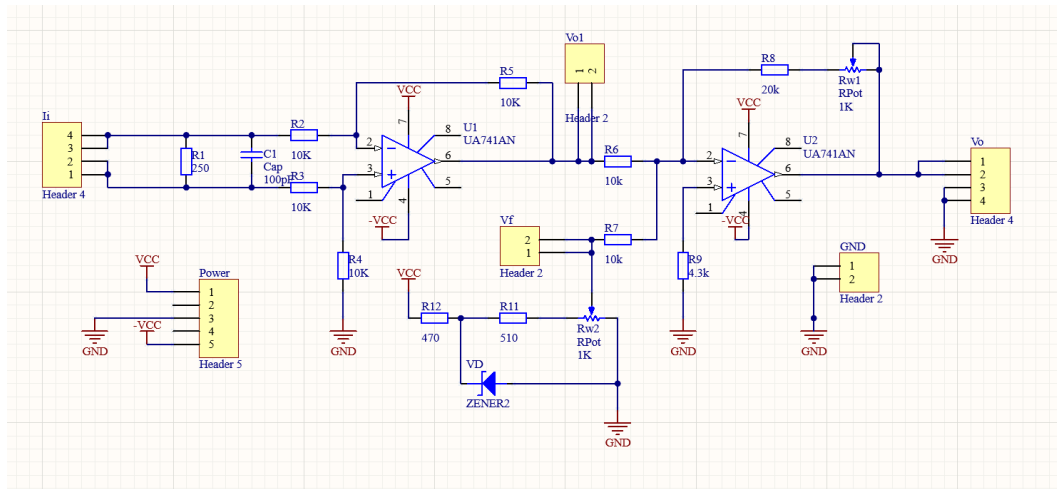


图 5: 电流电压转换实验原理图

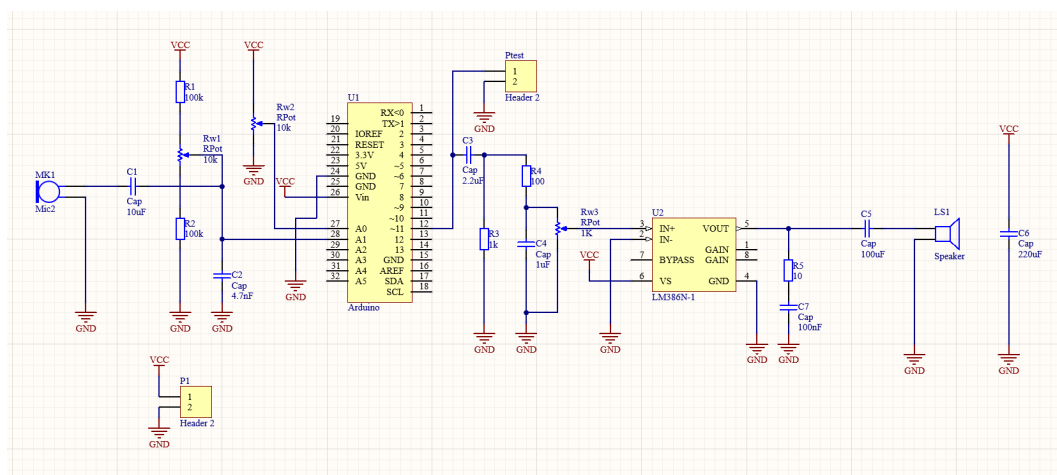


图 6: Arduino 音频处理实验原理图

七、 PCB 设计

电流电压转换实验 pcb 设计如图7；Arduino 音频处理实验 pcb 设计如图8。

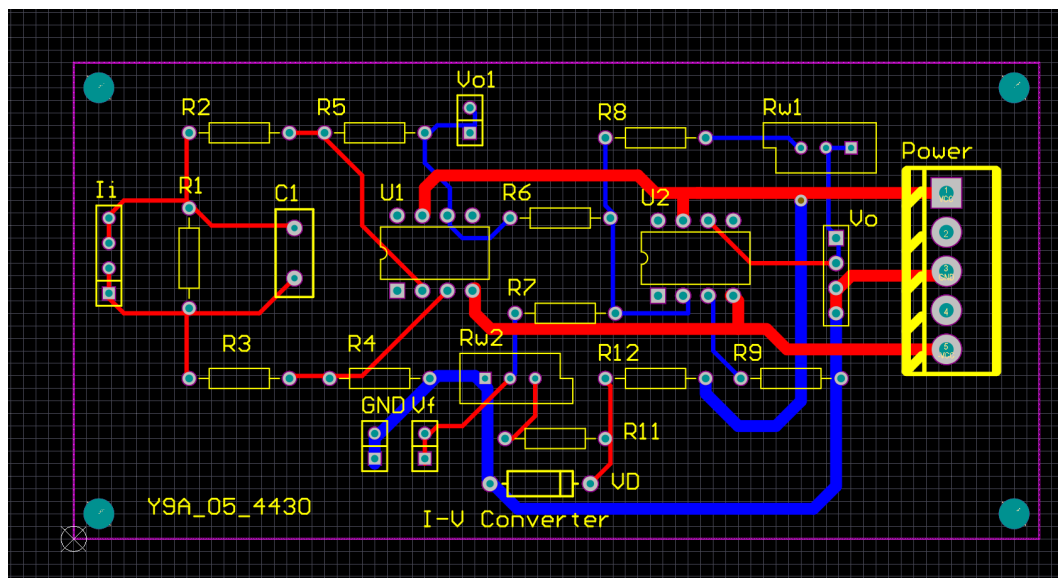


图 7: 电流电压转换实验 pcb 图

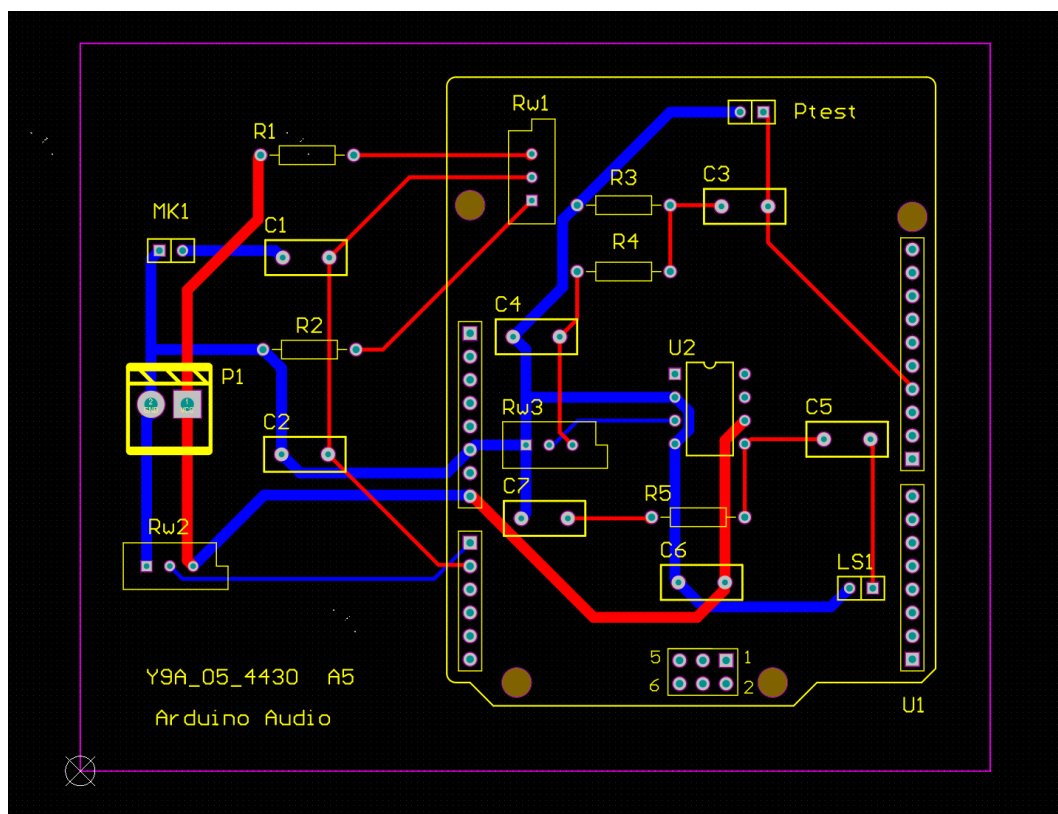


图 8: Arduino 音频处理实验 pcb 图

最终加工时选择了同组另一位同学的 PCB 进行加工，其 PCB 设计如图9。

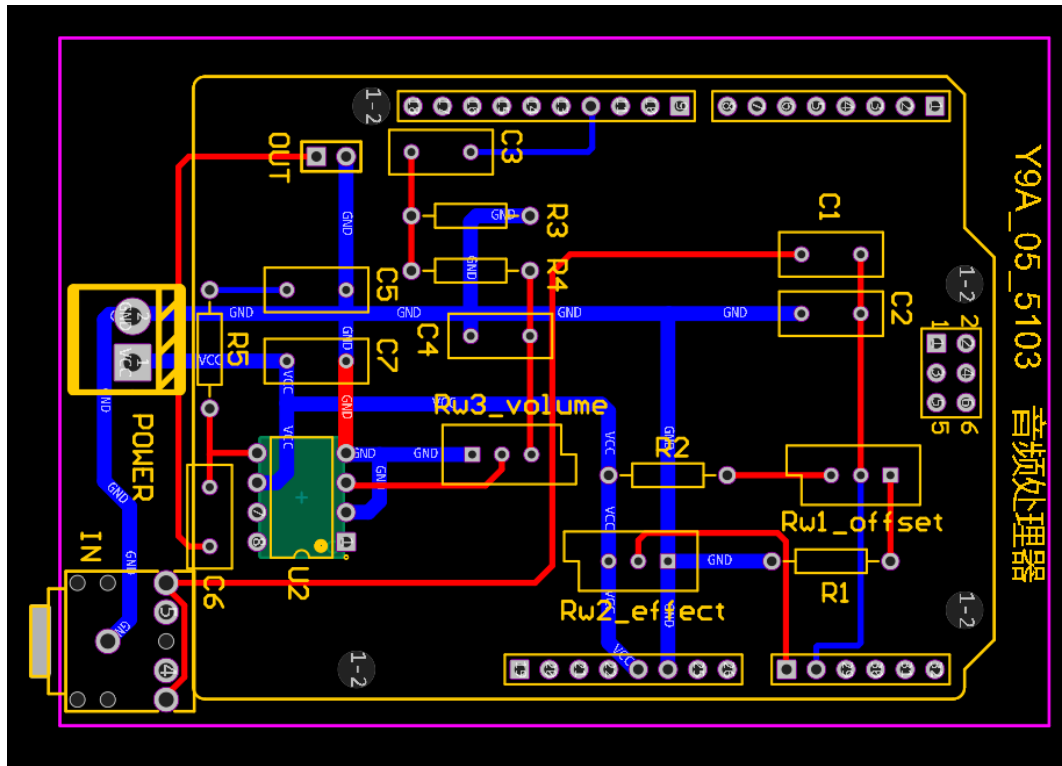


图 9: 用于打板的 Arduino 音频处理实验 pcb 图

八、 程序代码

Arduino 音频处理实验程序代码如下:

```

1 #define sbi(PORT,bit) (PORT |= 1 << (bit))
2 #define cbi(PORT,bit) (PORT &= ~(1 << (bit)))
3
4 const int pwmOutPin = 11;
5 const byte halfMax = 127;
6 boolean div2, selChan;
7
8 volatile boolean adcSample;
9 volatile byte potSampler, analogSample;
10 int indexCounter, valAnalog, valPot;
11 byte pwmBuf, ringbuff[512]; // 8位音频环形缓冲区
12
13 void setup() {
14     pinMode(pwmOutPin, OUTPUT); // 设置PWM引脚为输出模式
15
16     // 配置ADC预分频器 (分频因子64, ADC时钟=系统时钟/64)
17     cbi(ADCSRA, ADPS2);
18     sbi(ADCSRA, ADPS1);
19     sbi(ADCSRA, ADPS0);

```

```

20
21 // 配置ADC: 左对齐结果, 参考电压为AVcc
22 sbi(ADMUX, ADLAR);
23 sbi(ADMUX, REFS0);
24 cbi(ADMUX, REFS1);
25
26 // 设置ADC多路复用器为ADC0
27 cbi(ADMUX, MUX0);
28 cbi(ADMUX, MUX1);
29 cbi(ADMUX, MUX2);
30 cbi(ADMUX, MUX3);
31
32 // 配置Timer2为快速PWM模式, 非反相模式
33 cbi(TCCR2A, COM2A0);
34 sbi(TCCR2A, COM2A1);
35 sbi(TCCR2A, WGM20);
36 sbi(TCCR2A, WGM21);
37 cbi(TCCR2B, WGM22);
38
39 // 设置Timer2预分频器 (不分频, 时钟=系统时钟)
40 sbi(TCCR2B, CS20);
41 cbi(TCCR2B, CS21);
42 cbi(TCCR2B, CS22);
43
44 // 使能Timer2 PWM引脚 (Arduino D11 = PB3)
45 sbi(DDRB, 3);
46
47 // 禁用Timer0中断 (避免delay()等函数影响)
48 cbi(TIMSK0, TOIE0);
49 // 使能Timer2溢出中断
50 sbi(TIMSK2, TOIE2);
51
52 valAnalog = analogSample;
53 }
54
55 void loop() {
56 // 等待ADC采样完成
57 while (!adcSample);
58 adcSample = false;
59
60 // 用电位器值缩放延迟采样
61 valPot = (halfMax - ringbuff[indexCounter]) * potSampler / 255;
62 valAnalog = halfMax - analogSample;
63 valAnalog += valPot;
64

```

```

65 // 限制模拟值范围（防止溢出）
66 if (valAnalog < -halfMax) valAnalog = -halfMax;
67 if (valAnalog > halfMax) valAnalog = halfMax;
68
69 // 添加直流偏置，转换为无符号8位值
70 pwmBuf = halfMax + valAnalog;
71 ringbuff[indexCounter] = pwmBuf;
72 indexCounter++;
73
74 // 环形缓冲区索引回绕（0~511）
75 indexCounter &= 511;
76
77 // 将当前采样值写入PWM通道
78 OCR2A = pwmBuf;
79 }
80
81 // Timer2溢出中断服务程序：读取ADC值并切换通道
82 ISR(TIMER2_OVF_vect) {
83     PORTB |= 1; // 调试用：翻转PB0电平（可接示波器查看中断频率）
84     div2 = !div2; // 采样频率二分频
85
86     if (div2) {
87         selChan = !selChan;
88         if (selChan) {
89             potSampler = ADCH; // 读取电位器值（ADC1）
90             sbi(ADMUX, MUX0); // 切换多路复用器到ADC1
91         } else {
92             analogSample = ADCH; // 读取音频输入值（ADC0）
93             cbi(ADMUX, MUX0); // 切换多路复用器到ADC0
94             adcSample = true; // 标记ADC采样完成
95         }
96     }
97
98     sbi(ADCSRA, ADSC); // 启动下一次ADC转换
99 }

```

九、实验调试过程

9.1 Arduino 实时音频处理实验

9.1.1 电路调零校准

电路中设置了三个可调节的电位器 Rw1、Rw2、Rw3，其对应的功能分别为：Rw1 用于调节输入音频的直流耦合，Rw2 电位器直接接入 Arduino 的模拟端，用以给 Arduino 读取模拟值控制混响的延迟效果；Rw3 位于放大电路中运放的输入端，控制输出的音量大小。

首先调节 R_{w1} 使之处于 2.5V 附近，保证输入的直流耦合为供电的一半，可以使电路有最大的输入摆幅以防止失真（类似模拟电路中“静态工作点”的设置）。我们在代码中临时加上了串口输出读取电平值的功能，不断调节旋钮，使串口的输出达到 128 左右（Arduino 内部将模拟信号从 0-5V 映射为 0-255，因此 128 左右对应 2.5V），如图10。

调节完 R_{w1} 后，可自由调节 R_{w2} 、 R_{w3} ，以控制混响效果与音量。

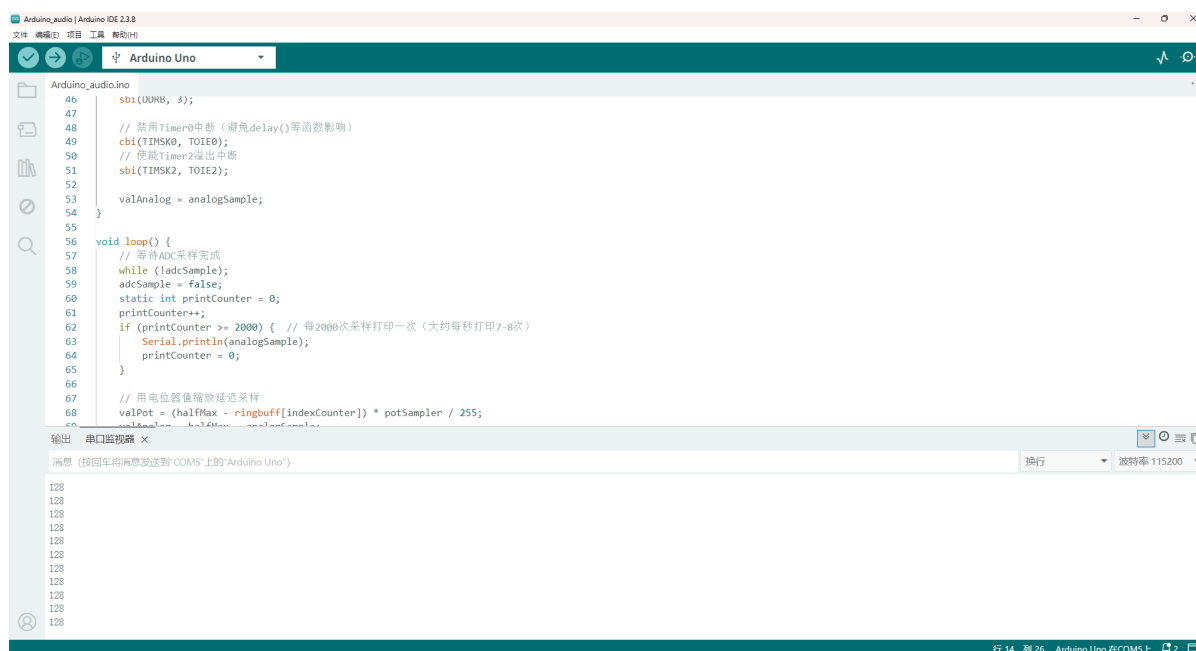


图 10: 调节 R_{w1} 偏置电压

9.1.2 输入设备更换

调试电路中，我们发现音频输出的人声很“闷”，就算把混响调到最低也很难分辨人讲话的具体内容；经老师提示后，我们将麦克风的输入替换为信号发生器的正弦波输入，听到了较为清晰的不同频率的声音；观察信号发生器中输出的峰峰值 V_{p-p} 在 1V 左右、最低能分辨的峰峰值也有 500mV，而用示波器观察到麦克风引起的输入的震动仅在几十到几百毫伏之间，因此最终确定是麦克风的灵敏度较低引起的电压变化不明显，和同组同学讨论后决定更换麦克风。

查阅资料可知，麦克风元器件（咪头）的灵敏度用分贝 (dB) 表示，实验室常见的灵敏度在 -52dB 左右，所对应的输出电压波动只有几十毫伏，因此导致波动不明显、Arduino 进行 ADC 采样后出现失真；更换为自行购买的 -26dB 的麦克风后测试，发现效果得到明显提升。

另外我们还尝试使用 3.5mm 耳机孔接入手机进行音频输入，也取得了良好的音频输出效果；调节混响的电位器，可以明显感受到“回声”的感觉。最终采用了这种方式进行验收。

9.1.3 输出级低通滤波器调整

接入音乐播放时，可以感受到，男声的声音基本正常，但是女声音高高的部分就会存在一定的失真；仔细分析参考资料中给的电路设计，发现输出级的低通滤波电容 $1\mu F$ 设置过大，根据低通滤波器截止频率计算公式

$$f = \frac{1}{2\pi RC} = 1592Hz$$

正常人说话频率大概在 200-3000Hz，可见原电路截止频率过低，因此导致高音出现失真问题。将电容更换为 $0.1\mu F$ 后，理论截止频率变为原先的十倍，声音明显更加清晰。

十、实验结果展示

10.1 电流电压转换实验

加工得到的 pcb 如图11；完成焊接后，对电路首先进行校准，分别调节控制零点的电位器和控制放大倍数的电位器，使输入电流为 4mA 时输出电压为 0V；输入电流为 20mA 时输出电压为 10V；完成调零后，选取 4mA-20mA 中间的几组数据进行测量，结果如表1；进行线性拟合所得结果见图12：

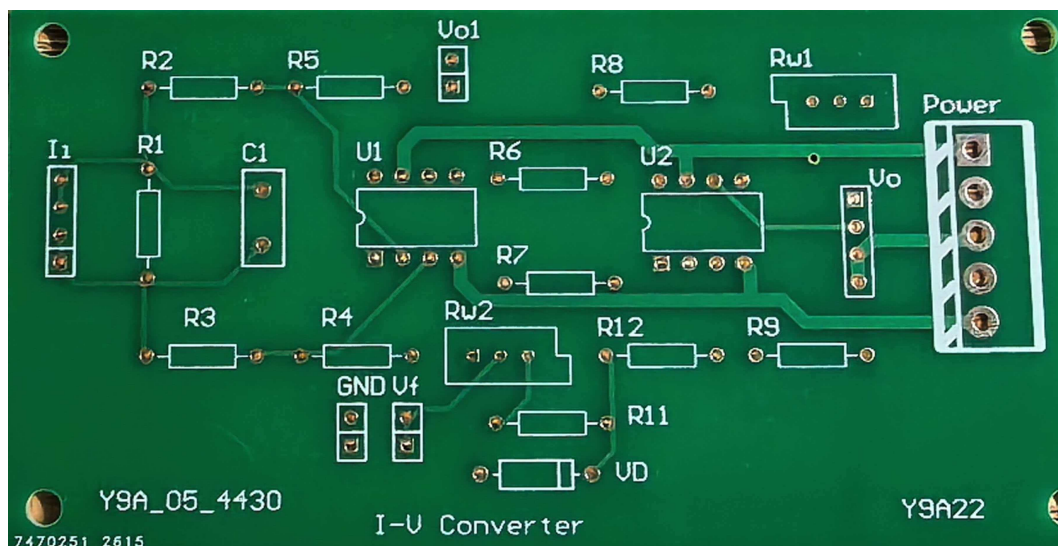


图 11: 电流电压转换实验 pcb 实物图

表 1: 电流电压转换电路测量

I_i	V_{o1}	V_o
4mA	-979.26mV	4.400mV
8mA	-1.9565V	2.5154V
12mA	-2.9290V	5.0362V
16mA	-3.9006V	7.5020V
20mA	-4.8886V	10.012V

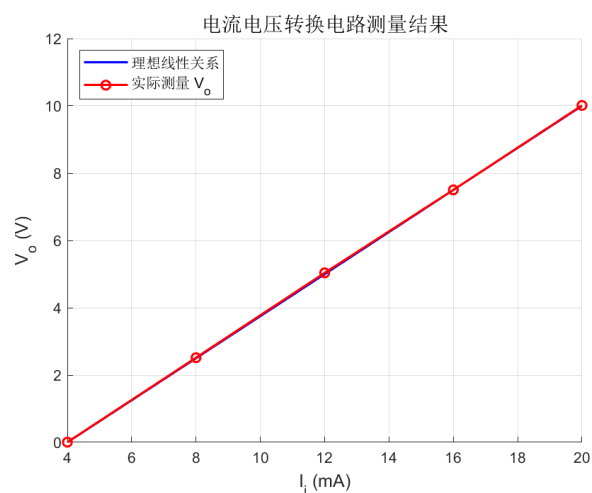


图 12: 电流电压转换电路测量结果

10.2 Arduino 实时音频处理实验

加工得到的 pcb 如图13；焊接完成的电路如图14。

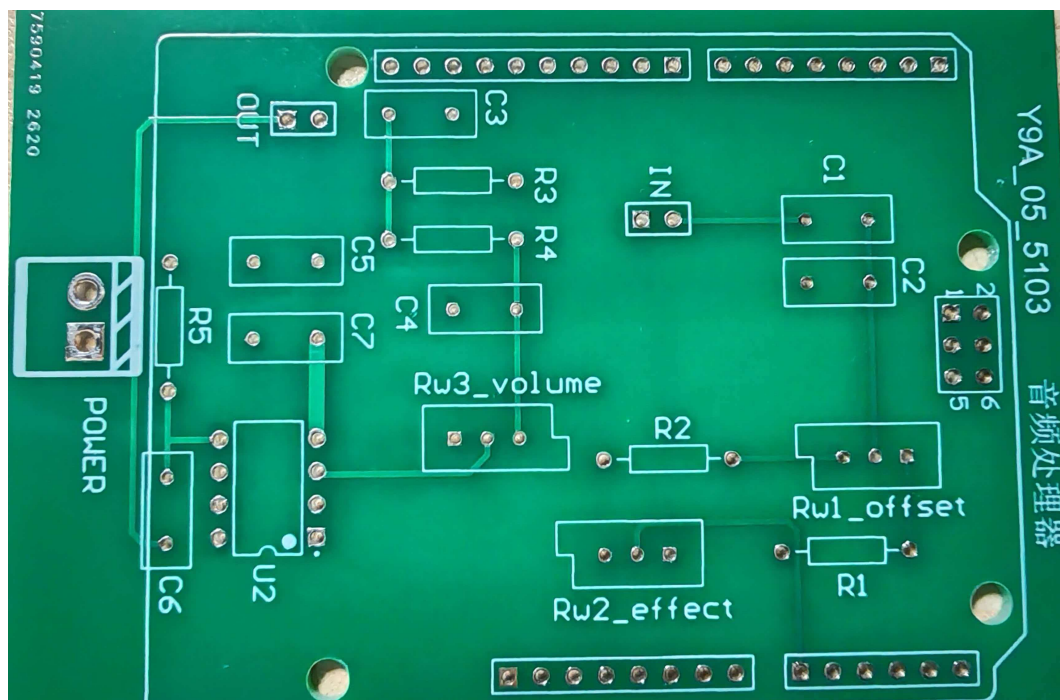


图 13: pcb 空板

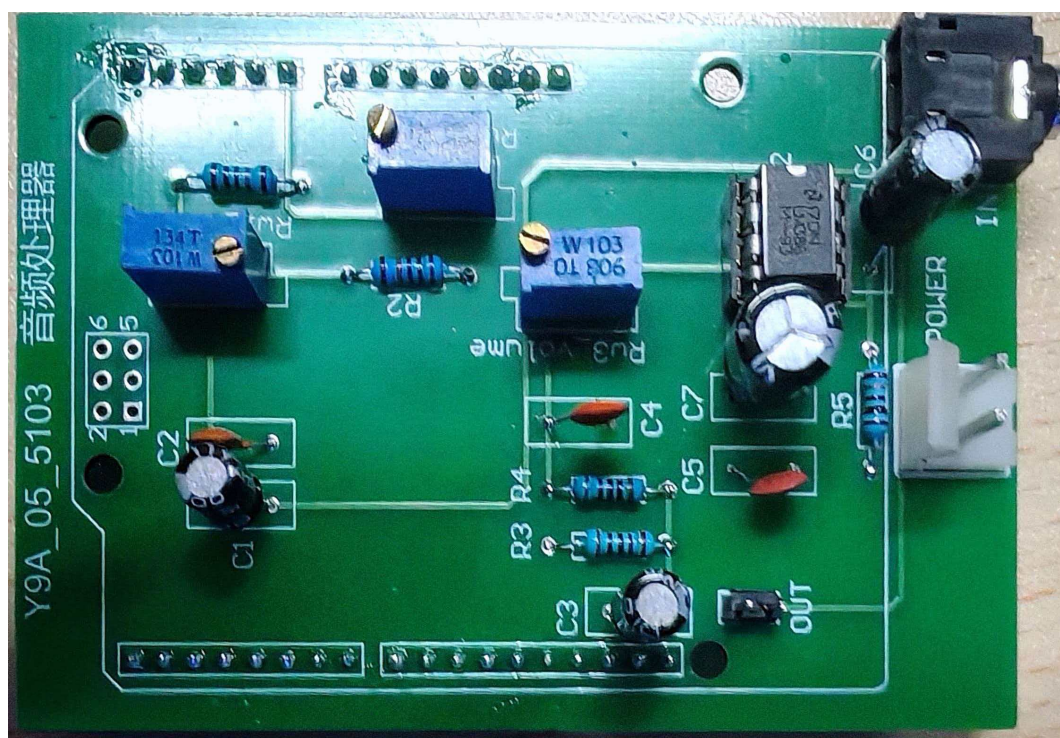


图 14: 焊接完成的实物电路

10.2.1 示波器波形观察

首先不接入任何输入，将输出接入示波器中，观察到 Arduino 输出的 62.5kHz 空 PWM 载波波形（图15a）。

再接入频率 1kHz、峰峰值为 600mV 的信号发生器产生的正弦波，观察输出的外轮廓包络线，为峰峰值 2.5V 左右、频率 1kHz 的正弦波（图15b）；在时间轴上放大观察波形细节，发现包络线中间部分确实是由 Arduino 产生的 62.5kHz 的 PWM 波填充的（图15c）。

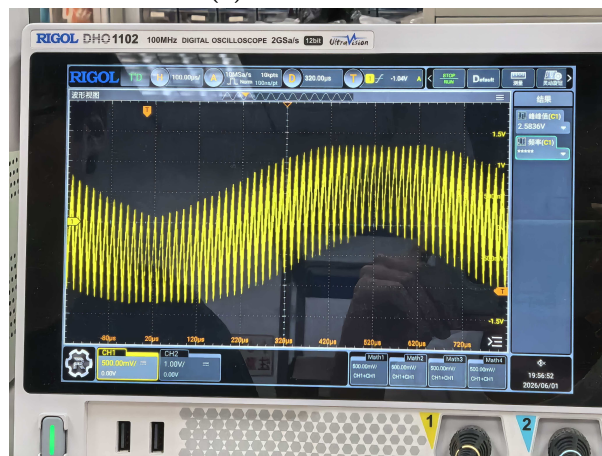
调大输入的幅度，可以看到输出的峰峰值也等比例放大；但如果一直调大输入正弦信号的峰峰值，就可以看到输出被明显的”削顶”失真（图15d），这是 Arduino 的输出只能为 0-5V 决定的。



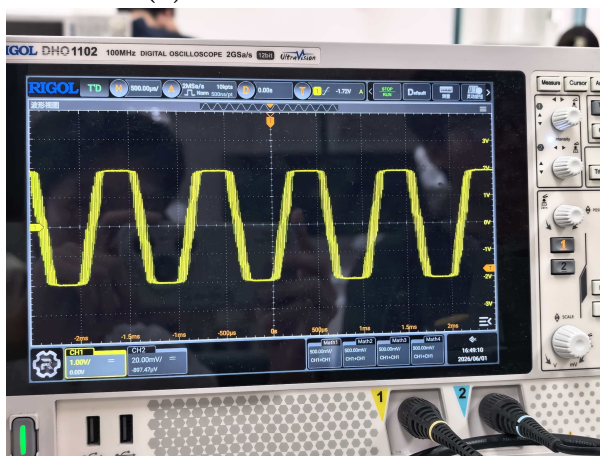
(a) 空 PWM 载波



(b) 输入 1kHz 正弦波（包络）



(c) 输入 1kHz 正弦波（细节）



(d) 输出削顶失真

图 15: 示波器波形观察

将真实的音乐音频作为输入，观察到输出波形如图16，整体形状较为合理，发声基本正常。

10.2.2 混响效果感受

接入麦克风时，调节控制混响效果的电阻，可以听到类似“回声”的效果；接入音频时，这种混响的效果更为明显：将混响调节到比较大的水平时，可以明显感受到前后的音交叠在一起，给人一种“3D 环绕”的演唱会听感。

另外，由于电路的采样频率不是很高、输出有一定的底噪，播放的音乐存在一定的失真度，碰巧有一种“老式 DVD”的颗粒感，此时播放复古音乐反而别有一番效果。



图 16: 输入音乐音频

十一、 实验结果分析

11.1 电流电压转换实验

观察表1中数据和图12中的拟合结果，发现电路的输出电压与输入电流基本成线性关系，拟合出来的 I-V 关系图（红线）也几乎与理论值（蓝线）重合，故可得出结论，该电路功能基本满足要求；此外，该电路 PCB 布局较为合理、走线美观，属于较为合格的设计。

11.2 Arduino 实时音频处理实验

该实验整体难度较大，经过我和同组同学几周的调试最终的结果也仍存在诸多细节上的小问题：

1. 使用高灵敏度麦克风输入时，仍然难以清晰分辨人说话的咬字。更换为-26dB 的麦克风后，输入的幅度对于该电路来说仍然偏小，导致 Arduino 采样量化后丢失细微的电压变化、因此导致输出的声音难以分辨咬字。（但-26dB 已经是市面上能找到的较为灵敏的麦克风了；结合该项目参考资料中的原理图，音频输入端没有接麦克风所需要的上拉偏置电阻，合理推测原项目中就是使用耳机孔而非麦克风作为音频输入）

2. 输出级存在饱和失真的问题，将输出的音量调大就会在示波器中观察到输出的波形被“削顶”失真，这是由于 Arduino 的输出电压只有 0-5V、负载能力有限导致的；和同组同学讨论后，我们认为，更为合理的设计应当是：在 Arduino PWM 输出与 LM386 之间增加一级电压跟随器作为缓冲，隔离后级对 PWM 输出的负载效应，同时可调节分压使信号幅度匹配 LM386 的输入范围。

十二、 讨论总结、体会 (1071 字)

本次实验的两个子实验分别侧重模拟电路与数模混合系统，但核心都离不开运放应用、电阻网络和 PCB 布线这些基础内容，让我有机会把课堂上学到的模电知识在工程场景中实际运用了一次。

电流电压转换实验完成得相对顺利。按照两级运放的思路，第一级用 250Ω 取样电阻将 4-20mA 电流转换为-1到-5V 电压，第二级通过反相放大和电平移位得到 0-10V 输出。焊接完成后，调零和调满度花费了一些时间，但最终实测数据与理论直线吻合良好，电路功能基本满足要求。通过这个实验，我也理解了工业控制中采用电流信号传输的原因——电流信号在长线传输中不受压降影响，4mA 作为活零点还可以方便地检测断线故障，比直接传输电压信号更可靠。

Arduino 音频处理实验的调试过程则复杂得多。第一次上电后声音模糊不清，起初怀疑是程序问题，后来用示波器逐级排查，才定位到多个方面的原因：麦克风灵敏度偏低，输出电压波动仅有几十毫伏；输出级低通滤波电容取值过大，导致 2kHz 以上的频率成分被过度衰减；Arduino PWM 输出幅度限于 0-5V，后级 LM386 增益稍高就容易出现削顶失真。前后更换麦克风、调整滤波电容、反复校准偏置电压，花费了数周时间，最终才得到了比较清晰的混响效果。这个过程虽然辛苦，但也让我体会到硬件调试与纸上分析有着本质区别——很多问题仅靠理论推导难以预见，必须借助仪器逐点测量、逐级排查，这正是工程实践不同于理论学习的地方。

团队合作方面，这次和同组的徐莘宁同学配合得很好，一人侧重原理图设计和程序编写，另一人主要负责 PCB 布局与硬件调试，遇到困难就一起分析讨论。有几次两人一起调试到晚上，虽然疲惫，但确实有一种共同克服困难后的充实感，这种经历在平时的理论课程中不太容易获得。

回顾整个实验，我最大的收获是认识到：工程设计不能想当然，参考资料上的电路图看似完善，但实际搭建时往往会遇到各种意料之外的问题——因为器件的参数不同、供电条件不同、布线方式不同，很多细节是电路图上不会标出的。只有亲自动手做一遍、出错一遍、修正一遍，才算真正理解了一个电路的工作原理。这种反复实践、不断纠错的过程，可能比最终的实验结果更有价值。此外，遇到问题时保持耐心、沉下心来逐项排查，与队友及时沟通、互相分担——这些事情说起来简单，但真正面对多个变量交织的故障时并不容易做到，也是本次实验让我收获最大的地方。